Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение

высшего образования «Челябинский государственный университет»

(ФГБОУ ВО «ЧелГУ»)

Задача № 4

по дисциплине «Информатика»

Проверил:

Преподаватель
Лихачёв К. А.
Автор работы:
Иванов Н. Е.
Студент группы: МПМ-101

Формулировка задания

1 Реализовать сравнение представление матрицы смежности и списка смежности

Нужно создать две матрицы(матрицу смежности и список смежности) и потом их сравнить друг с другом

- **Решение задания**

Далее, после этого абзаца, представлен код программы, написанной на языке программирования Java.

1 интерфейс графа

```
public interface Graph {  14 usages  4 implementations
    void addEdge(int u, int v);  2 usages  4 implementations
    boolean hasEdge(int u, int v);  2 usages  4 implementations
    List<Integer> getNeighbors(int u);  2 usages  4 implementations
    int getVertexCount();  no usages  4 implementations
    int getEdgeCount();  no usages  4 implementations
    String getRepresentationType();  no usages  4 implementations
}
```

2 создание матрицы смежности

```java
static class AdjacencyMatrixGraph implements Graph {  2 usages
    private final boolean[][] matrix;  6 usages
    private final int V;  6 usages
    private int E;  3 usages

    public AdjacencyMatrixGraph(int vertices) {  2 usages
        if (vertices <= 0) throw new IllegalArgumentException("Количество вершин должно быть > 0");
        this.V = vertices;
        this.matrix = new boolean[V][V];
        this.E = 0;
    }

    @Override  2 usages
    public void addEdge(int u, int v) {
        validate(u, v);
        if (!matrix[u][v]) {
            matrix[u][v] = true;
            matrix[v][u] = true; // неориентированный граф
            E++;
        }
    }

    @Override  2 usages
    public boolean hasEdge(int u, int v) {
        validate(u, v);
        return matrix[u][v];
    }
```

```java
    }

    @Override  2 usages
    public List<Integer> getNeighbors(int u) {
        validate(u);
        List<Integer> neighbors = new ArrayList<>();
        for (int i = 0; i < V; i++) {
            if (matrix[u][i]) neighbors.add(i);
        }
        return neighbors;
    }

    @Override public int getVertexCount() { return V; }  no usages
    @Override public int getEdgeCount() { return E; }  no usages
    @Override public String getRepresentationType() { return "Матрица смежности"; }  no usages

    private void validate(int... vertices) {  3 usages
        for (int v : vertices) {
            if (v < 0 || v >= V) throw new IndexOutOfBoundsException("Недопустимая вершина: " + v);
        }
    }
}
```

3. создание списка смежности

```java
static class AdjacencyListGraph implements Graph {  2 usages
    private final List<List<Integer>> adj;  6 usages
    private final int V;  5 usages
    private int E;  3 usages

    public AdjacencyListGraph(int vertices) {  2 usages
        if (vertices <= 0) throw new IllegalArgumentException("Количество вершин должно быть > 0");
        this.V = vertices;
        this.adj = new ArrayList<>(V);
        for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
        this.E = 0;
    }

    @Override  2 usages
    public void addEdge(int u, int v) {
        validate(u, v);
        List<Integer> listU = adj.get(u);
        List<Integer> listV = adj.get(v);
        if (!listU.contains(v)) {
            listU.add(v);
            listV.add(u);
            E++;
        }
    }

    @Override  2 usages
    public boolean hasEdge(int u, int v) {
        validate(u, v);
        return adj.get(u).contains(v);
    }
```

```java
    @Override  2 usages
    public List<Integer> getNeighbors(int u) {
        validate(u);
        return new ArrayList<>(adj.get(u));
    }

    @Override public int getVertexCount() { return V; }  no usages
    @Override public int getEdgeCount() { return E; }  no usages
    @Override public String getRepresentationType() { return "Список смежности"; }  no usages

    private void validate(int... vertices) {  3 usages
        for (int v : vertices) {
            if (v < 0 || v >= V) throw new IndexOutOfBoundsException("Недопустимая вершина: " + v);
        }
    }
}
```

4. создания метода сравнения по количеству вершин и количеству ребер

```java
public static void main(String[] args) {
    final int V = 1000;
    final int E = 5000;
    final Random rand = new Random( seed: 42);

    Graph matrixGraph = new AdjacencyMatrixGraph(V);
    Graph listGraph = new AdjacencyListGraph(V);
```

5. генерация одинакого набора ребер

```java
List<int[]> edges = new ArrayList<>();
for (int i = 0; i < E; i++) {
    int u = rand.nextInt(V);
    int v = rand.nextInt(V);
    if (u != v) edges.add(new int[]{u, v});
}

System.out.println("СРАВНЕНИЕ ПРЕДСТАВЛЕНИЙ ГРАФА");
System.out.println("Вершин: " + V + " | Рёбер: ~" + E);
System.out.println("—".repeat( count: 50));
```

6. Далее у нас идет добавление ребер, проверка наличия ребер и получение соседей

```java
benchmark( name: "Добавление ребра", edges, ( Graph g,  int[] e) -> g.addEdge(e[0], e[1]), matrixGraph, listGraph);


benchmark( name: "Проверка ребра", edges, ( Graph g,  int[] e) -> g.hasEdge(e[0], e[1]), matrixGraph, listGraph);


List<Integer> checkVertices = new ArrayList<>();
for (int i = 0; i < 100; i++) checkVertices.add(rand.nextInt(V));
benchmark( name: "Получение соседей", checkVertices, ( Graph g,  Integer v) -> g.getNeighbors(v), matrixGraph, listGraph);
```

7. создание универсального бенчмарка для списка и матрицы

```java
private static <T> void benchmark(String name, List<T> inputs, BiConsumer<Graph, T> operation, Graph matrix, Graph list) {
    long t1 = System.nanoTime();
    for (T input : inputs) operation.accept(matrix, input);
    long timeMatrix = System.nanoTime() - t1;


    long t2 = System.nanoTime();
    for (T input : inputs) operation.accept(list, input);
    long timeList = System.nanoTime() - t2;

    System.out.printf("%-22s | Матрица: %,8d ns | Список: %,8d ns | Δ: %.1f%%\n",
            name, timeMatrix, timeList, (1.0 - (double)timeList/timeMatrix) * 100);
}
```

- **Результаты**

Результаты программы:

```
СРАВНЕНИЕ ПРЕДСТАВЛЕНИИ ГРАФА
Вершин: 1000 | Рёбер: ~5000
──────────────────────────────────────────────────

Добавление ребра       | Матрица: 1 429 700 ns | Список: 3 677 500 ns | Δ: -157,2%
Проверка ребра         | Матрица: 1 533 900 ns | Список: 1 569 000 ns | Δ: -2,3%
Получение соседей      | Матрица: 2 295 400 ns | Список:  313 200 ns | Δ: 86,4%

ТЕОРЕТИЧЕСКАЯ СЛОЖНОСТЬ
Операция           | Матрица      | Список       | Примечание
──────────────────────────────────────────────────────────────────────

Добавление ребра   | O(1)         | O(deg(v))    | в списке поиск дубля O(deg)
Проверка ребра     | O(1)         | O(deg(v))    |
Получение соседей  | O(V)         | O(1)         | возврат готового списка
Память             | O(V²)        | O(V + E)     | матрица всегда V² bool
```

Как мы видим матрица выигрывает в проверках/добавлении, список выигрывает в получении соседей и памяти.